

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бекэнд разработка

Отчет

Домашнее задание №5

«Межсервисное взаимодействие через RabbitMQ»

Выполнил:
Дементьев Антон
K3341

Проверил:
Добряков Д. И

Санкт-Петербург

2026 г.

Задача

В лабораторной работе №2 («Реализация микросервисов») межсервисные события доставлялись по HTTP: outbox-диспетчер отправителя вызывал POST /internal/v1/events получателя. В разделе 9 отчёта ЛР 2 это было заявлено как временное упрощение относительно архитектуры, спроектированной в ДЗ 4, где для событийного взаимодействия предусмотрен брокер сообщений. Задача ДЗ 5 — подключить и настроить брокер сообщений (RabbitMQ) и перевести на него межсервисное взаимодействие по событиям, не меняя бизнес-логику обработчиков и внешние контракты системы.

Ход работы

За основу взят код ЛР 2 (семь сервисов и API Gateway, npm-монорепозиторий, общий пакет packages/common). Транспорт событий заменён с HTTP на RabbitMQ; транзакционные таблицы outbox_events/inbox_events и идемпотентность обработчиков (processOnce) сохранены без изменений — заменена только доставка сообщения от издателя к получателю. За основу взят пример от преподавателя (kantegory/mentoring, 26_express_rabbitmq): сервисы обмениваются сообщениями через очереди RabbitMQ, читая библиотеку amqplib напрямую, без дополнительных фреймворков поверх неё.

1. Брокер сообщений

Выбран RabbitMQ (образ rabbitmq:3-management — включает веб-интерфейс управления). Контейнер rabbitmq добавлен в docker-compose.yml: порт 5672 — протокол AMQP, порт 15672 — management UI (<http://localhost:15672>, логин/пароль jobsearch/jobsearch), healthcheck rabbitmq-diagnostics -q ping, данные — в volume dbs/rabbitmq-data. Каждый сервис теперь зависит от rabbitmq (condition: service_healthy), как и от postgres. Адрес брокера передаётся сервисам переменной окружения RABBITMQ_URL=amqp://jobsearch:jobsearch@rabbitmq:5672 (общая для всех сервисов, задана в x-common-env). На стороне Node.js добавлена зависимость amqplib (и её типы @types/amqplib).

2. Обмен событиями: топология

В пакете packages/common добавлен модуль mq.ts, инкапсулирующий работу с RabbitMQ. Все события публикуются в один topic-обмен jobsearch.events (durable) с routing key, равным типу события (например vacancy.upserted). Получатель объявляет свою durable-очередь и привязывает её к нужным routing key — издателю не нужно знать состав подписчиков, в отличие от прежней HTTP-версии, где outbox.ts содержал явную карту «тип события → список сервисов-получателей».

Событие (routing key)	Издатель	Очередь получателя	Что происходит
identity.email_requested	identity-service	notification.identity-events	«отправка» письма с кодом подтверждения, запись EmailLog
vacancy.upserted	vacancy-service	recommendation.catalog-events	обновление вакансии в индексе подбора
vacancy.deleted	vacancy-service	recommendation.catalog-events	удаление вакансии из индекса

resume.upserted	resume-service	recommendation.catalog-events	обновление резюме в индексе подбора
resume.deleted	resume-service	recommendation.catalog-events	удаление резюме из индекса

Синхронные межсервисные вызовы (сервисные токены, close-then-check при удалении вакансии/резюме, batch-lookup через reference/vacancy/resume/identity, ежечасная сверка целостности Application Service) по-прежнему выполняются через HTTP-клиент ServiceClient (client.ts) — заменена только асинхронная доставка событий, для которой HTTP не даёт гарантий доставки и требует ручного учёта получателей.

3. Публикация: outbox → RabbitMQ

Событие, как и раньше, пишется в таблицу outbox_events в той же транзакции, что и изменение бизнес-данных (publishEvent). Фоновый планировщик startOutboxDispatcher (packages/common/src/outbox.ts) каждые 2 секунды выбирает неотправленные строки и публикует их в обмен jobsearch.events функцией publishEnvelope() из mq.ts — вместо прежнего цикла по списку получателей с HTTP-вызовом POST /internal/v1/events. При ошибке публикации (брокер недоступен) попытка откладывается с экспоненциальным back-off (attempts, next_attempt_at), как и в HTTP-варианте; строка помечается published_at только после успешной публикации в обмен. Сообщения публикуются с persistent: true, чтобы RabbitMQ сохранял их на диск и не терял при перезапуске брокера.

Соединение с брокером устанавливается лениво при первой публикации/подписке и переустанавливается с повтором (до 30 попыток с интервалом 2 с), по той же схеме, что и подключение к PostgreSQL при старте сервиса — если контейнер rabbitmq поднимается медленнее сервисов, они дождутся его, а не упадут.

4. Подписка и обработка событий

Общая функция startEventConsumer() (mq.ts) объявляет durable-очередь получателя, привязывает её к нужным routing key на обмене jobsearch.events, включает ограничение prefetch = 10 (сервис не забирает из очереди больше 10 неподтверждённых сообщений разом) и подписывается с ручным подтверждением (ack). Её используют два новых модуля:

- services/notification/src/consumer.ts — очередь notification.identity-events, обрабатывает identity.email_requested; логика (отправка «письма», запись EmailLog) перенесена без изменений из прежнего HTTP-обработчика POST /internal/v1/events;
- services/recommendation/src/consumer.ts — очередь recommendation.catalog-events, обрабатывает vacancy.*/resume.*; логика обновления индекса (upsertVacancy/deleteVacancy/upsertResume/deleteResume) перенесена без изменений из прежнего внутреннего контроллера.

Оба потребителя, как и раньше, вызывают processOnce(dataSource, event_id, ...): id события записывается в inbox_events в той же транзакции, что и результат обработки, поэтому повторная доставка одного и того же события (переподтверждение после сбоя, повторная попытка) не выполняет побочный эффект дважды. HTTP-эндпоинты POST /internal/v1/events в notification-service и recommendation-service удалены — их роль полностью выполняют подписчики RabbitMQ.

5. Надёжность и обработка сбоев

Подтверждение сообщения (ack) отправляется только после успешного завершения обработчика — если сервис упадёт до этого момента, RabbitMQ доставит сообщение

повторно после переподключения (redelivery), а идемпотентность на стороне `inbox_events` исключит повторный эффект. Если обработчик выбрасывает ошибку (например, временная недоступность БД получателя), сообщение переотправляется в ту же очередь со счётчиком попыток в заголовке `x-attempt`; после пяти неудачных попыток (настраивается параметром `maxAttempts`) сообщение уходит в `dead-letter` очередь `<очередь>.dead` через отдельный `fanout`-обмен `<очередь>.dlx` — так «ядовитое» сообщение не блокирует очередь бесконечно и остаётся доступным для ручного разбора в `management UI`, а не теряется.

Деградация получателя (`Recommendation` или `Notification` остановлены) не блокирует издателя: события продолжают писаться в `outbox` и публиковаться в `RabbitMQ`, накапливаются в `durable`-очереди на диске брокера и будут обработаны, как только получатель снова подключится — то же свойство, что было в `HTTP`-версии (там события накапливались в `outbox_events` на стороне издателя), но теперь буферизация происходит и на стороне брокера.

6. Изменения в конфигурации

Файл	Изменение	Назначение
<code>docker-compose.yml</code>	добавлен сервис <code>rabbitmq</code> (<code>rabbitmq:3-management</code> , <code>healthcheck</code> , порты <code>5672/15672</code>); <code>RABBITMQ_URL</code> в <code>x-common-env</code> ; <code>rabbitmq</code> — в <code>depends_on</code> каждого сервиса	брокер сообщений и его адрес для всех сервисов
<code>package.json</code>	добавлены зависимости <code>amqplib</code> и <code>@types/amqplib</code>	AMQP-клиент для <code>Node.js/TypeScript</code>
<code>packages/common/src/mq.ts</code>	новый модуль: подключение к брокеру с повтором, <code>publishEnvelope()</code> , <code>startEventConsumer()</code> с ретраями и <code>dead-letter</code>	общая инфраструктура публикации/подписки для всех сервисов
<code>packages/common/src/outbox.ts</code>	<code>startOutboxDispatcher()</code> публикует событие в <code>RabbitMQ</code> вместо <code>HTTP</code> -вызова получателей; таблицы <code>outbox_events/inbox_events</code> и <code>processOnce</code> не изменены	смена транспорта без изменения гарантий доставки
<code>services/notification</code> , <code>services/recommendation</code>	новый <code>consumer.ts</code> ; <code>HTTP</code> -обработчик <code>POST /internal/v1/events</code> удалён из контроллеров	получатели читают события из <code>RabbitMQ</code>
<code>README.md</code>	добавлен раздел «ДЗ 5. RabbitMQ», обновлён раздел «Отличия от ДЗ 4»	документация

7. Как проверить

Требуется `Docker` и `Docker Compose` (образ `rabbitmq:3-management` загружается автоматически). Если запущена более ранняя версия `lab2` без `RabbitMQ` — предварительно `docker compose down`, чтобы применить новый `docker-compose.yml`.

- распаковать архив и перейти в папку `lab2`;
- выполнить `docker compose up -d --build` (первая сборка — несколько минут, включая загрузку образа `RabbitMQ`);

- проверить `docker compose ps` — должно быть 10 контейнеров в состоянии Up (postgres, rabbitmq, api-gateway и 7 сервисов);
- открыть `http://localhost:8080/api/v1/health` — ожидается `{"status":"ok","version":"1.0.0"}`;
- открыть `http://localhost:15672` (jobsearch/jobsearch) — на вкладке Exchanges виден jobsearch.events, на вкладке Queues — notification.identity-events и recommendation.catalog-events с привязками (Bindings) к нужным routing key;
- прогнать сценарий Postman (см. ниже) и убедиться, что счётчики Message rates на вкладке Queues в management UI отражают публикацию и обработку событий (например, при создании/удалении вакансий и резюме).

Проверка в Postman не изменилась по сравнению с ЛР 2: используется та же коллекция `postman/JobSearch_API.postman_collection.json` (40 запросов, 4 папки) и окружение `postman/JobSearch_gateway.postman_environment.json` с переменной `base_url = http://localhost:8080/api/v1`.

- Postman → File → Import → выбрать оба файла из папки postman/ и выбрать окружение «JobSearch gateway (ЛР 2)»;
- убедиться, что система запущена (см. выше) и `http://localhost:8080/api/v1/health` отвечает `status: ok`;
- открыть коллекцию «JobSearch API» и нажать Run — запросы выполнятся по порядку (0. Подготовка данных → 1. Соискатель → 2. Работодатель → 3. Проверка результата), тесты внутри запросов проверят статусы и поля ответов автоматически;
- коды подтверждения email коллекция получает автоматически через `GET /api/v1/dev/mailbox?email=...` — событие `identity.email_requested` к этому моменту должно быть доставлено через RabbitMQ и обработано `notification-service`; если письмо не появляется, стоит проверить очередь `notification.identity-events` в management UI (Ready/Unacked/Dead) и логи `docker compose logs notification-service`;
- успешное прохождение коллекции (как и в ЛР 2) подтверждает, что публичный контракт не изменился; то, что рекомендации подбираются по вакансиям и резюме, созданным по ходу сценария, подтверждает, что индекс Recommendation действительно обновляется событиями `vacancy.*/*resume.*`, доставленными теперь через RabbitMQ, а не по HTTP.

Вывод

Межсервисное взаимодействие по событиям в проекте ЛР 2 переведено с HTTP-доставки на брокер сообщений RabbitMQ: добавлен и настроен брокер (`docker-compose`, `healthcheck`, `management UI`), реализована публикация событий из `outbox` в `topic`-обмен `jobsearch.events` и подписка получателей на `durable`-очереди с ручным подтверждением, повторной доставкой при ошибке обработчика и `dead-letter` очередью после исчерпания попыток. Транзакционный `outbox/inbox` и идемпотентность обработчиков, заложенные в ЛР 2, не изменились — заменён только транспорт, как и предполагалось разделом 9 отчёта ЛР 2. Тем самым устранено последнее из трёх упрощений относительно архитектуры ДЗ 4 в части событийного взаимодействия; публичный контракт системы (50 операций шлюза, коды ошибок) и сценарий проверки в Postman не изменились.